

放弃RR选择RC，通过for update理解MySQL锁定机制

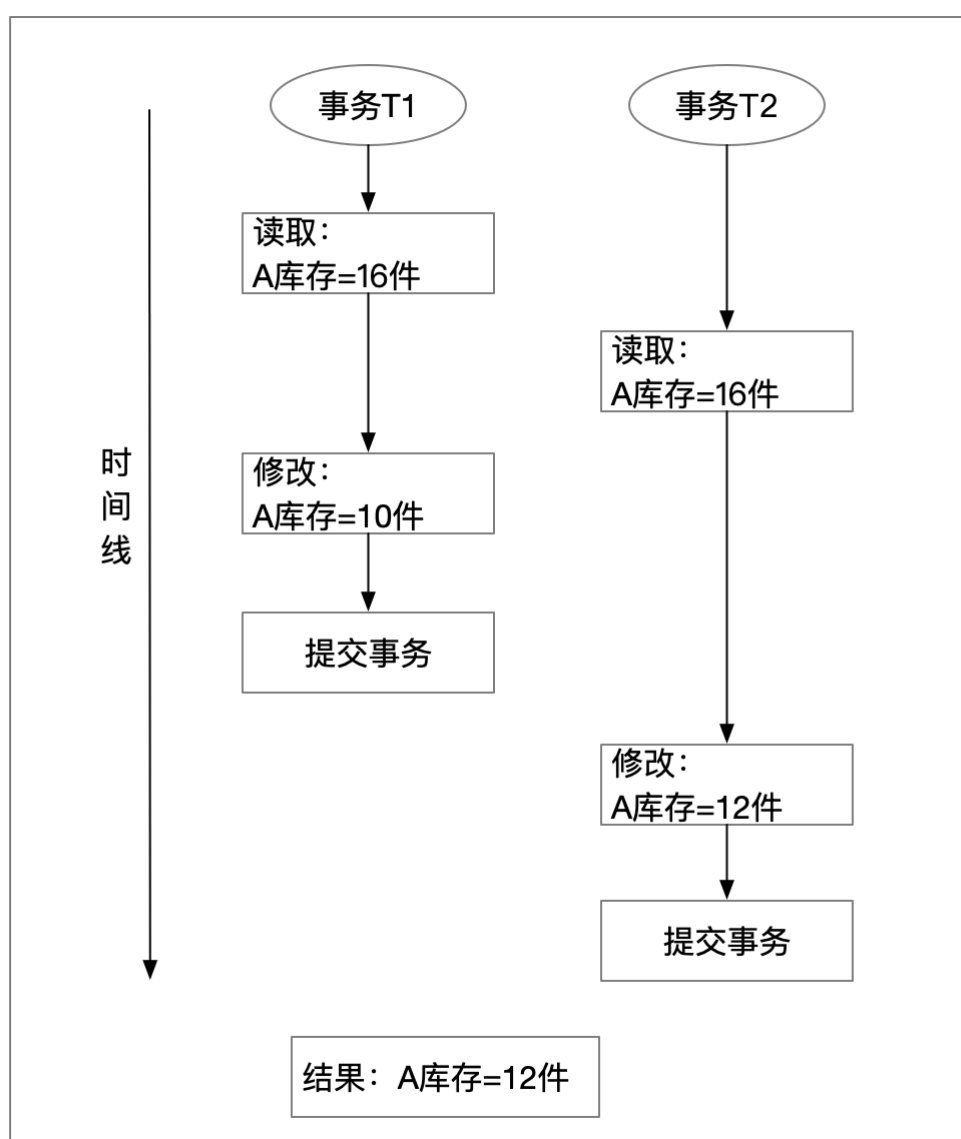
事务并发问题

修改丢失：两个事务同时修改一个数据，发生了修改覆盖问题。

脏读：是指发生在一个事务读取了另一个事务改写但尚未提交的数据时。如果改写在稍后被回滚了，则第一个事务获取的数据就是无效的。A事务读取到了B事务尚未提交的数据【原子性】

不可重复读：A事务执行2次相同查询查到了不同数据【一致性】，避免不可重复读需要锁行就行，不可重复读重点在于update和delete

幻读：是指发生在事务（T1）读取了几行数据，接着另一个并发事务T2插入了一些数据时。在随后的查询中，第一个事务T1就会发现多了一些原本不存在的记录。A事务读取到了B事务插入的数据【隔离性】，避免幻读则需要锁表，幻读的重点在于insert



通过上图所示，我们了解到，两个事务T1、T2同时读入同一数据并修改，T2提交的结果会破坏或覆盖T1提交的结果，导致T1的修改丢失。

数据库隔离性

READ UNCOMMITTED: 读未提交(RU), 脏读, 不可重复读、幻读都存在, 所有读不加锁, 读到的数据都是最新数据, 性能最好, 所有写加锁, 写完释放

READ COMMITTED: 读已提交(RC), 解决了脏读, 存在不可重复读, 幻读问题。oracle默认隔离级别

REPEATABLE READ: 可重复读(RR), 解决了脏读, 不可重复读, 存在幻读。适用mvcc技术实现不加锁的读操作。mysql默认隔离级别

SERIALIZABLE: 串行化(S), 全部解决, 但是并发效率不高

隔离级别	脏读	不可重复读	幻读
读未提交 Read uncommitted	√	√	√
读已提交 Read committed	X	√	√
可重复读 Repeatable read	X	X	√
可串行化 Serializable	X	X	X

CSDN @mister_guo

RR与RC的区别

在RR中, 快照会在事务中第一次查询语句执行时生成, 只有在本事务中对数据进行更改才会更新快照。也就是说, 如果在本事务中已经执行了一次select, 此时其它事务执行了更改数据的操作并已提交, 那么你在本事务再次select时, 你是看不到其它事务所做的更改。

事务流程/事务名	事务A	事务A
	1、begin开启事务	2、begin开启事务
	3、select * (查到2条数据)	4、select * (查到2条数据)
	7、select * (查到2条数据)	5、insert/commit 一条数据
	8、commit	6、select * (查到3条数据)
	9、select * (查到3条数据)	

在RC中, 每次读取都会重新生成一个快照, 总是读取行的最新版本。也因此事务中每次select也可以看到其它已commit事务所做的修改。

事务流程/事务名	事务A	事务A
	1、begin开启事务	2、begin开启事务
	3、select * (查到2条数据)	4、select * (查到2条数据)
	7、select * (查到3条数据)	5、insert/commit 一条数据
	8、commit	6、select * (查到3条数据)
	9、select * (查到3条数据)	

在数据库的RC这种隔离级别中，还支持半一致读，一条update语句，如果where条件匹配到的记录已经加锁，那么InnoDB会返回记录最近提交的版本，由MySQL上层判断此是否真的需要加锁。

为什么大厂放弃RR选择RC？

大厂之所以使用RC，主要原因有二，其一是提升并发，其二是减少死锁。

为什么RC要比RR并发度好？

因为RC在加锁过程中，是不需要添加Gap Lock(间隙锁)和Next-Key Lock(临键锁)的，只需要对要修改的记录添加行级锁就行了。另外，RC还支持半一致读，可以大大的减少了更新语句时行锁的冲突。对于不满足更新条件的记录，可以提前释放锁，提升并发度。

为什么RC可以减少死锁发生？

因为RR会增加Gap Lock(间隙锁)和Next-Key Lock(临键锁)，这就使得锁的粒度变大了，那么就容易导致死锁的概率增大（占地太宽了，别人占不到地方了）。

通过for update理解MySQL锁定机制

数据库的锁，在不同的事务隔离级别下，是采用了不同的机制的。在MySQL中，有三种类型的锁，分别是Record Lock、Gap Lock和Next-Key Lock。

Record Lock表示记录锁，锁的是索引记录。

- 共享锁（shared lock）也叫读锁、S锁，数据被某个事务获取了共享锁之后，还可以被其它事务获取共享锁，但是不能被其它事务获取排他锁。
- 排他锁（exclusive Lock）也叫写锁、X锁，数据被某个事务获取之后就不能被其它事务获取任何锁

举个例子，当一个事务执行了下面这条语句：

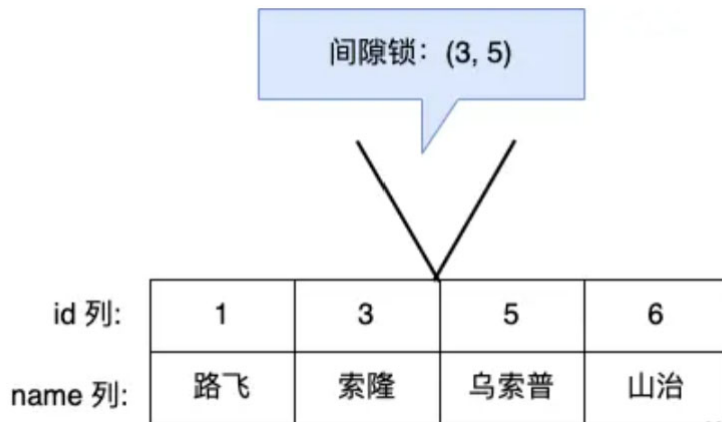
```
mysql > begin;
mysql > select * from t_test where id = 1 for update;
```

就是对t_test表中主键id为1的这条记录加上X型的记录锁，这样其他事务就无法对这条记录进行修改了。



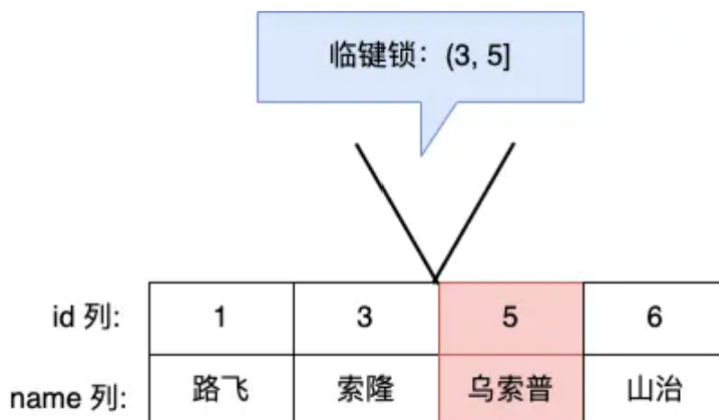
Gap Lock是间隙锁，在索引记录之间的间隙中加锁，或者是在某一条索引记录之前或者之后加锁，并不包括该索引记录本身。gap lock的机制主要是解决可重复读模式下的幻读问题

假设，表中有一个范围 id 为 (3, 5) 间隙锁，那么其他事务就无法插入 id = 4 这条记录了，这样就有效的防止幻读现象的发生



Next-Key Lock是临键锁，是 Record Lock + Gap Lock 的组合，锁定一个范围，并且锁定记录本身。既然是组合，那我们可以理解为只有Gap Lock存在的情况下才会有Next-Key Lock

假设，表中有一个范围 id 为 (3, 5] 的 next-key lock，那么其他事务即不能插入 id = 4 记录，也不能修改 id = 5 这条记录。



在RC中，只会对索引增加Record Lock，不会添加另外两种锁。

在RR中，为了解决幻读问题，三种锁都支持。

for update行锁和表锁

for update是一种行级锁，又叫排它锁，一旦用户对某个行施加了行级加锁，则该用户可以查询也可以更新被加锁的数据行，其它用户只能查询但不能更新被加锁的数据行。如果其它用户想更新该表中的数据行，则也必须对该表施加行级锁。即使多个用户对一个表均使用了共享更新，但也不允许两个事务同时对一个表进行更新，真正对表进行更新时，是以独占方式锁表，一直到提交或复原该事务为止。行锁永远是独占方式锁。

只有当出现如下之一的条件，才会释放共享更新锁：

- 1、执行提交 ([COMMIT](#)) 语句
- 2、退出数据库
- 3、程序停止运行

概念和用法

通常情况下，select语句是不会对数据加锁，妨碍影响其他的DML和DDL操作。同时，在多版本一致读机制的支持下，select语句也不会被其他类型语句所阻碍。

而select ... for update 语句是我们经常使用手工加锁语句。在数据库中执行select ... for update ,大家会发现会对数据库中的表或某些行数据进行锁表，在mysql中，如果查询条件带有主键，会锁行数据，如果没有，会锁表。

由于InnoDB预设是Row-Level Lock，所以只有明确的指定主键，MySQL才会执行Row lock (只锁住目标行)，否则MySQL将会执行Table Lock (将整个表给锁住)。

举个例子: 假设有张表user，里面有 id 和 name 两列，id是主键。

例1: (明确指定主键，并且数据真实存在，row lock)

```
SELECT * FROM user WHERE id=3 FOR UPDATE;

SELECT * FROM user WHERE id=3 and name='Tom' FOR UPDATE;
```

例2: (明确指定主键，但数据不存在，无lock)

```
SELECT * FROM user WHERE id=0 FOR UPDATE;
```

例3: (主键不明确，table lock)

```
SELECT * FROM user WHERE id<>3 FOR UPDATE;

SELECT * FROM user WHERE id LIKE '%3%' FOR UPDATE;
```

例4: (无主键，table lock)

```
SELECT * FROM user WHERE name='Tom' FOR UPDATE;
```

注意：

- 1、FOR UPDATE仅适用于InnoDB，且必须在事务处理模块(BEGIN/COMMIT)中才能生效。
- 2、Myisam 只支持表级锁，InnerDB支持行级锁 添加了(行级锁/表级锁)锁的数据不能被其它事务再锁定，也不被其它事务修改。是表级锁时，不管是否查询到记录，都会锁定表。